

Kafka, CDC и потоковая обработка

Лекция 7

Kafka

CDC

Structured Streaming

Lakehouse

Где мы в курсе

HDFS отвечал на вопрос, где физически хранить большие данные.

Spark дал нам распределенный движок для обработки этих данных.

Lakehouse добавил управляемые таблицы, версии, качество и воспроизводимость.

Сегодня обсуждаем вход в эту систему: поток событий, изменения из баз данных и Spark Structured Streaming.

Проблема: бизнес меняется быстрее ночной обработки

Ночная пакетная обработка удобна, когда достаточно обновлять отчеты один раз в сутки.

Но заказ могут создать, оплатить, отменить и вернуть задолго до следующего ночного запуска.

Если аналитическая платформа узнает об этом слишком поздно, витрины, алерты и ML-признаки начинают жить вчерашним днем.

Batch и streaming не конкурируют

Batch

Хорош для больших регулярных пересчетов.

Проще отлаживать и воспроизводить.

Часто используется для тяжелых витрин.

Streaming

Хорош, когда события приходят постоянно.

Снижает задержку между фактом и аналитикой.

Требует аккуратной работы со временем и сбоями.

В зрелой платформе обычно есть оба подхода: batch пересчитывает, streaming поддерживает свежесть.

Событие и состояние - разные вещи

Состояние заказа

В таблице лежит одна актуальная строка.

Например: `order_id = 42, status = cancelled`.

Это удобно для отчетов и витрин.

История событий

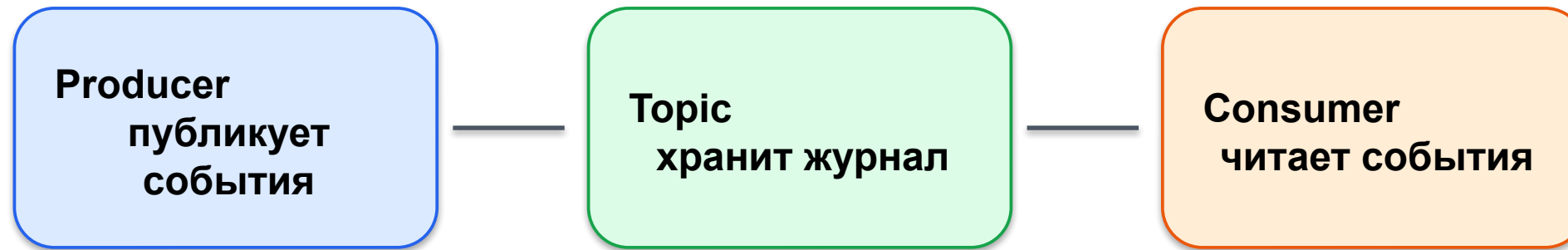
Заказ создали, оплатили, потом отменили.

Каждое изменение сохраняется как отдельный факт.

Из этих фактов мы собираем актуальное состояние.

Streaming почти всегда начинается с журнала фактов, а не с готовой таблицы состояния.

Kafka простыми словами



Kafka - это надежный распределенный журнал событий. Она не заменяет Spark и не строит витрины сама.

Topic, partition, offset



Partition дает параллелизм, offset дает позицию чтения.

Один поток могут читать разные системы

Один и тот же topic может одновременно кормить витрины, ML-признаки, мониторинг и антифрод.

Каждая consumer group хранит собственную позицию чтения.

Поэтому новый потребитель может появиться позже и читать события независимо от остальных.

CDC: как изменения из базы становятся событиями

CDC расшифровывается как Change Data Capture - захват изменений данных.

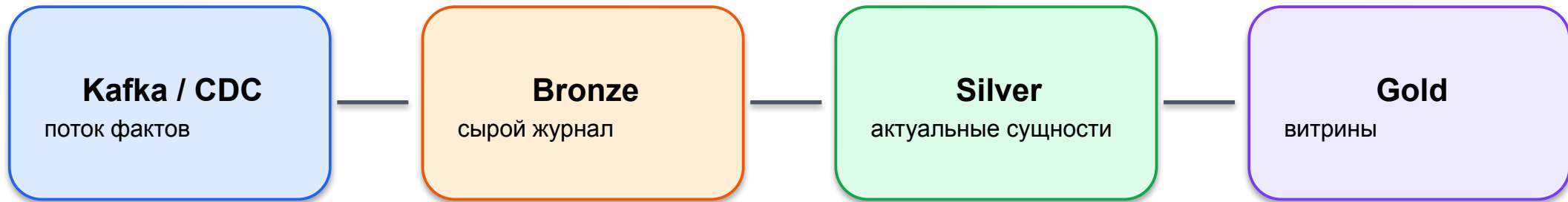
Вместо полной выгрузки таблицы каждые несколько минут мы читаем поток изменений: insert, update и delete.

Так аналитическая система узнает не только о новых заказах, но и об оплатах, отменах, возвратах и исправлениях.

Как выглядит CDC-событие

```
{  
  "op": "u",  
  "order_id": 42,  
  "before": {"status": "created"},  
  "after": {"status": "paid"},  
  "event_ts": "2026-04-25T09:05:00Z"  
}
```

Куда приземлять поток



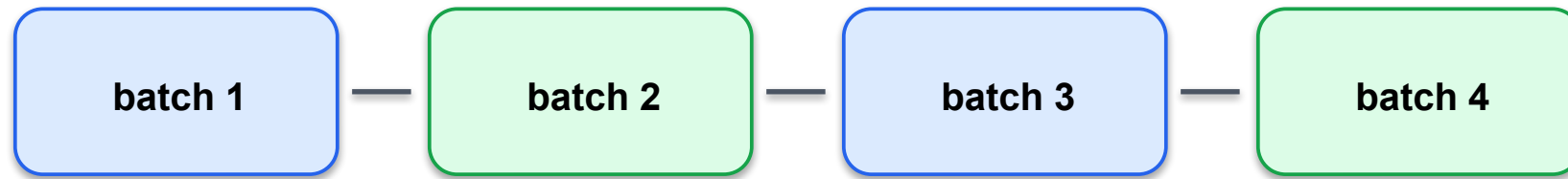
Structured Streaming - это Spark для потоков

В PySpark потоковые данные выглядят почти как обычный DataFrame.

Мы читаем их через `readStream`, описываем transformations и запускаем запись через `writeStream`.

Главное отличие в том, что такой DataFrame потенциально бесконечный, поэтому Spark должен хранить прогресс и состояние запроса.

Micro-batch: поток как серия маленьких batch



Spark часто обрабатывает поток короткими микробатчами: забрал новые события, посчитал, записал результат, сохранил прогресс.

Checkpoint - память потокового запроса

Checkpoint хранит, какие данные источник уже отдал Spark.

Там же может лежать состояние для агрегатов, дедупликации и соединений.

Если streaming job упал, Spark использует checkpoint, чтобы продолжить работу с правильного места.

Удалить checkpoint - почти то же самое, что стереть память запроса.

Дубликаты - нормальная часть потоковых систем

При сбое consumer может перечитать часть событий.

Это лучше, чем потерять данные, но downstream должен уметь пережить повтор.

Поэтому событиям нужен event_id, а записи в таблицы должны быть идемпотентными: повторный запуск не должен менять результат.

В потоках есть два времени

Event time

Когда событие произошло в бизнесе.

Например, заказ оплатили в 09:05.

Именно по этому времени обычно считают метрики.

Ingest time

Когда событие приехало в аналитику.

Например, сообщение дошло в 09:25.

Это время важно для мониторинга задержек.

Поздно пришедшее событие не обязательно является новым с точки зрения бизнеса.

Окна и водяной знак (watermark)

Окно отвечает на вопрос: за какой интервал времени мы считаем метрику.

Watermark отвечает на другой вопрос: как долго мы готовы ждать опоздавшие события.

Если ждать бесконечно, состояние streaming job будет расти бесконечно.

Если ждать слишком мало, часть поздних событий не попадет в агрегаты.

Когда поток помнит прошлое

Дедупликация

Spark должен помнить уже
виденные event_id.

Агрегации

Spark хранит счетчики по
ключам и временным окнам.

Join

Spark может держать одну
сторону соединения в
состоянии.

Stateful streaming мощный, но требует контроля размера состояния.

Что на самом деле означает *exactly once*

В реальной системе важно не красивое обещание, а итоговый эффект.

Если событие перечитали два раза, витрина не должна удвоить заказ.

Для этого нужны *checkpoint*, уникальные ключи событий и запись, которую можно безопасно повторить.

Чаще всего проектируют *exactly-once effect*: результат выглядит так, будто событие применили один раз.

Схема в потоке - это контракт

В пакетной обработке плохая схема ломает один запуск.

В потоковой обработке плохая схема может остановить постоянно работающий pipeline и несколько downstream-систем.

Поэтому источник (producer) и потребитель (consumer) должны договориться о ключах, типах, обязательных полях и правилах изменения схемы.

Где здесь ClickHouse

ClickHouse обычно не заменяет Kafka, Spark или Lakehouse.

В ETL он чаще выступает как быстрый аналитический слой для готовых витрин.

Типичный сценарий: Spark очищает и агрегирует данные, Lakehouse хранит воспроизводимую историю, а ClickHouse быстро отвечает на запросы дашбордов и аналитиков.

Чем ClickHouse отличается от обычной SQL-базы

В лекции по SQL мы думали о языке запросов. ClickHouse добавляет важный физический слой: как таблица лежит на диске и как она читается.

Он колоночный, поэтому хорошо считает агрегаты по большим таблицам.

Он любит крупные вставки пачками и плохо подходит для частых маленьких update по одной строке.

Главный движок таблиц - семейство MergeTree: там особенно важны ORDER BY, партиции и стратегия дедупликации.

Как использовать ClickHouse в ETL

Не грузите в ClickHouse хаотичный сырой поток, если потом не сможете объяснить качество данных.

Чаще безопаснее писать в него Gold-витрины: уже очищенные, проверенные и понятные таблицы.

Загрузка должна быть идемпотентной: повторный запуск ETL не должен удваивать строки.

Для CDC-сценариев часто используют ReplacingMergeTree или пересборку партиций, но выбор зависит от ключей, объема и требований к свежести.

Что мониторить в потоковом pipeline

Отставание (lag)

Насколько потребитель отстал от источника.

Свежесть

Насколько свежи данные в витрине.

Ошибки

Сколько событий не прошло схему или качество.

Размер состояния

Не растет ли состояние без контроля.

Streaming без мониторинга сложно отличить от работающей системы, которая тихо отстала на несколько часов.

Типовые ошибки

Запустить streaming job без checkpoint и удивиться дублям после рестарта.

Считать метрики по ingest time, хотя бизнес спрашивает про event time.

Хранить бесконечное состояние без watermark.

Писать append туда, где нужен upsert по ключу.

Не проверять схему входящих событий.

Что делаем на семинаре

Создадим локальный источник событий, который будет вести себя как учебный Kafka topic.

Запустим Structured Streaming и запишем события в Bronze-слой.

Проверим checkpoint: повторный запуск не должен перечитывать старые файлы.

Из CDC-событий соберем актуальное состояние заказов и отдельно разберем позднее событие.

В конце посмотрим, как такую Gold-витрину можно выгрузить в ClickHouse.

Ключевая мысль

Streaming - это не просто способ посчитать быстрее.

**Это способ надежно превратить
поток фактов в состояние данных.**